

Introduction

Case Study Overview: Simple Cloud-Based Application Deployment

This case study focuses on deploying a basic cloud-based application using Amazon Web Services (AWS), exploring fundamental cloud infrastructure tools to create a scalable and robust website hosting solution. The project walks through the process of building a simple HTML webpage, deploying it as a static website using Amazon S3 (Simple Storage Service), and establishing a backup server using an Amazon EC2 (Elastic Compute Cloud) instance.

AWS Cloud9, an online development environment, is used to create the HTML page, which is then hosted on an S3 bucket for public access. The S3 service is configured for static website hosting, ensuring easy accessibility for users. Furthermore, an EC2 instance is set up with the Apache web server as a backup hosting environment, providing redundancy in case the primary website faces downtime.

This case study demonstrates an accessible, cost-effective approach to deploying cloud applications using the AWS ecosystem, suitable for beginners and adaptable to more complex scenarios. The project highlights the use of key cloud services—S3 for storage and static hosting, EC2 for virtual machine deployment, and Cloud9 for development—showcasing how these tools combine to offer a highly available web application in the cloud.

Key Feature and Application: Highlighting the Unique Feature and Its Practical Use

The standout feature of this cloud-based deployment is the **dual hosting setup** that combines **Amazon S3 static website hosting** with an **Amazon EC2 backup server**. This approach ensures high availability, redundancy, and cost efficiency for hosting a simple web application.

Unique Feature: S3 Static Website Hosting with EC2 Backup

- **Amazon S3 Static Website Hosting:** S3 is primarily used for file storage, but it also offers a unique feature—static website hosting. This allows users to host a fully scalable, static website directly from an S3 bucket, eliminating the need for managing servers. It's ideal for static content like HTML, CSS, and JavaScript, offering an economical, serverless option for website deployment.
- **Amazon EC2 Backup Server:** EC2 instances provide dynamic server hosting, but in this case, EC2 acts as a backup server, running an Apache web server. If the primary S3 website becomes unavailable, the EC2 instance ensures continuous access to the same content. This creates a failover system, enhancing the site's reliability.

Practical Application:

- **Cost-Effective Hosting:** Hosting a website on S3 is significantly more cost-effective than running an always-on server, as users only pay for the storage and the bandwidth they use. S3 can handle traffic spikes effortlessly without needing manual intervention to scale server resources, making it ideal for simple websites that experience varying traffic levels.
- **High Availability and Redundancy:** The integration of an EC2 instance as a backup server provides a high-availability solution. In the event of downtime or issues with the S3-hosted website, the EC2 backup server ensures uninterrupted service. This is critical for maintaining user trust and preventing revenue loss due to site downtime.
- **Practical Use Case:** This architecture is particularly useful for small businesses, startups, or personal websites that need a low-cost, scalable solution with minimal maintenance. The EC2 backup ensures that even in rare cases where the S3 bucket is inaccessible, the site remains live, offering a seamless experience for visitors.

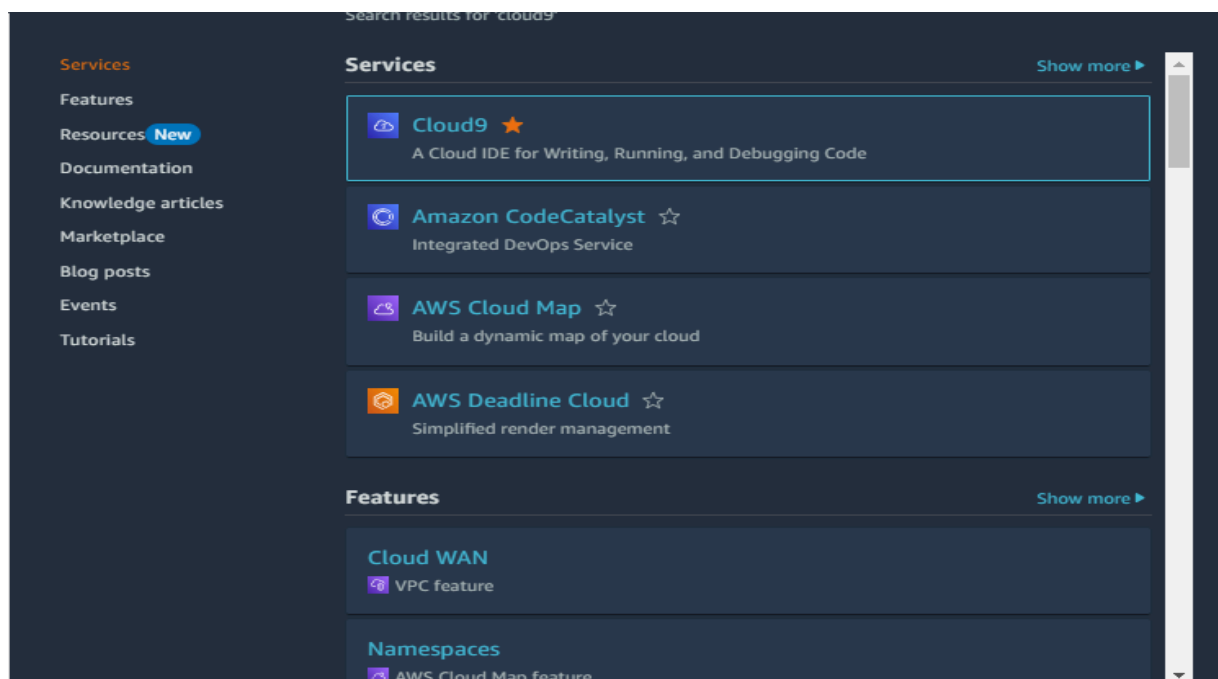
This dual-hosting strategy combines the best of both worlds—cost-efficient, scalable storage with S3 and reliable, on-demand hosting with EC2—making it a robust, cloud-based solution for modern web applications.

Explanation

1. Log in to your AWS Console

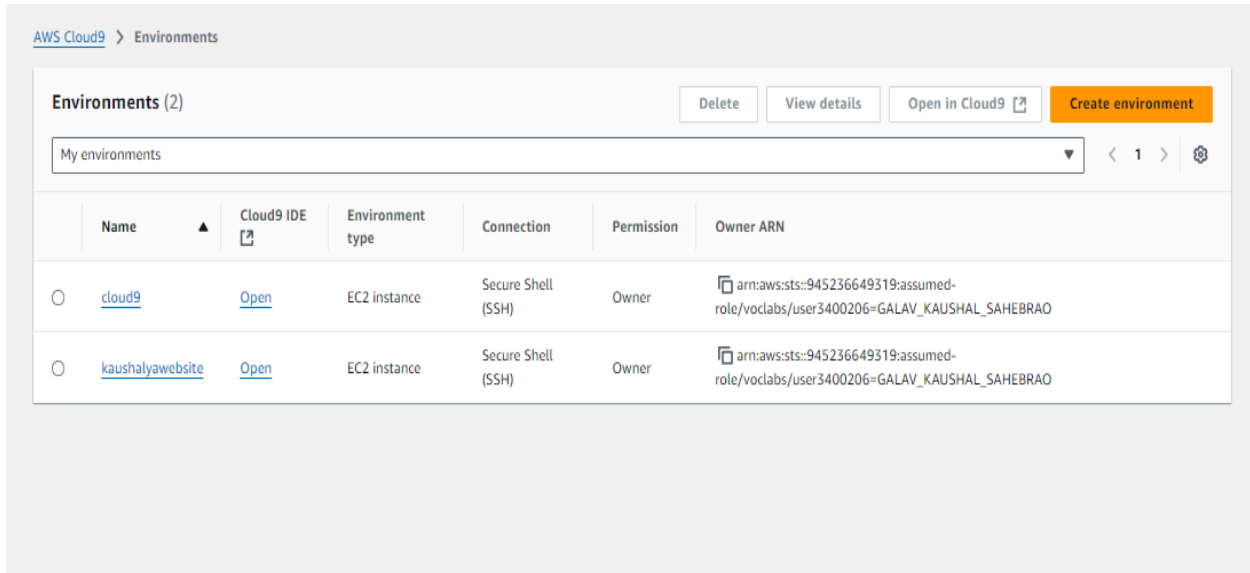
2. Navigate to AWS Cloud9

- **Find Cloud9 using the search bar:**
 - After logging in, locate the **search bar** at the top of the AWS Management Console.
 - Type **Cloud9** into the search bar, and from the results, click on **Cloud9** under "Developer Tools."
- **Open the Cloud9 service:**
 - This will bring you to the Cloud9 dashboard, where you can manage your Cloud9 environments.



3. Create a New Cloud9 Environment

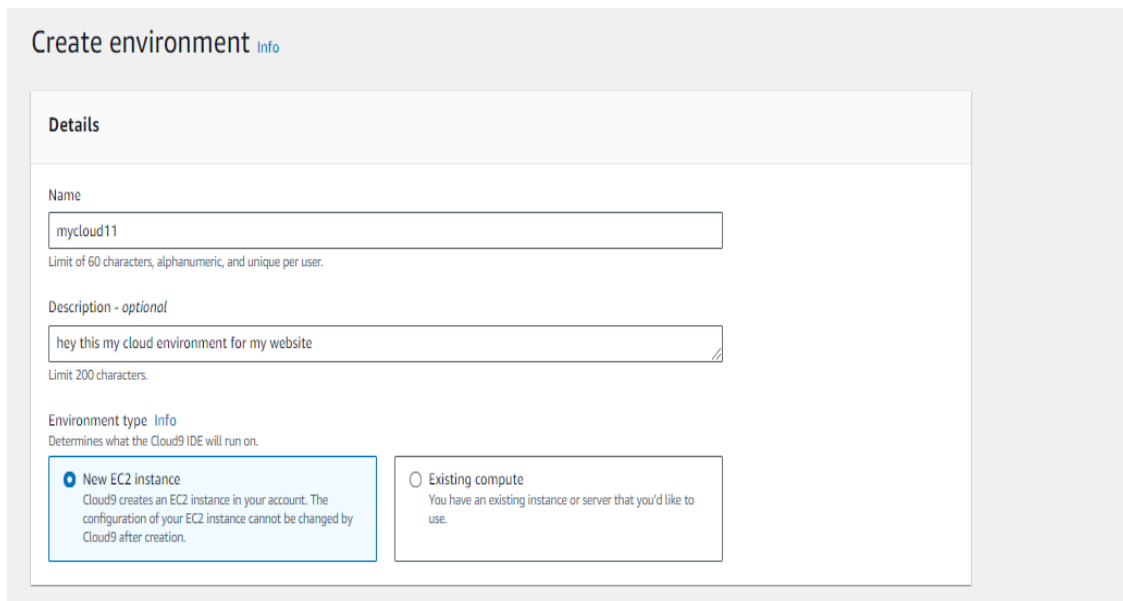
- **Start by creating an environment:**
 - Click on the **Create environment** button on the top right of the Cloud9 dashboard.



The screenshot shows the AWS Cloud9 'Environments' page. At the top, there's a breadcrumb 'AWS Cloud9 > Environments'. Below it, a header bar contains 'Environments (2)', 'Delete', 'View details', 'Open in Cloud9', and a prominent orange 'Create environment' button. A search bar labeled 'My environments' is present. The main content is a table with columns: Name, Cloud9 IDE, Environment type, Connection, Permission, and Owner ARN. Two environments are listed: 'cloud9' and 'kaushalyawebiste', both using EC2 instances with Secure Shell (SSH) connections and owned by 'arn:aws:sts::945236649319:assumed-role/voclabs/user3400206=GALAV_KAUSHAL_SAEHBRAO'.

	Name	Cloud9 IDE	Environment type	Connection	Permission	Owner ARN
☐	cloud9	Open	EC2 instance	Secure Shell (SSH)	Owner	arn:aws:sts::945236649319:assumed-role/voclabs/user3400206=GALAV_KAUSHAL_SAEHBRAO
☐	kaushalyawebiste	Open	EC2 instance	Secure Shell (SSH)	Owner	arn:aws:sts::945236649319:assumed-role/voclabs/user3400206=GALAV_KAUSHAL_SAEHBRAO

- **Give your environment a name:**
 - In the first step of creating the environment, you will need to **name** it. For example, name it **MyWebsiteEnv** (or any other name you prefer).
 - In the **Description** field, add something like "Environment for hosting my simple HTML page."



The screenshot shows the 'Create environment' form. It has a 'Details' section with three main fields: 'Name' (containing 'mycloud11'), 'Description - optional' (containing 'hey this my cloud environment for my website'), and 'Environment type'. The 'Name' field has a limit of 60 characters. The 'Description' field has a limit of 200 characters. The 'Environment type' section has two radio buttons: 'New EC2 instance' (selected) and 'Existing compute'. The 'New EC2 instance' option includes a note that Cloud9 creates an EC2 instance and its configuration cannot be changed after creation. The 'Existing compute' option includes a note that you must have an existing instance or server to use.

Create environment Info

Details

Name
mycloud11
Limit of 60 characters, alphanumeric, and unique per user.

Description - optional
hey this my cloud environment for my website
Limit 200 characters.

Environment type Info
Determines what the Cloud9 IDE will run on.

☒ **New EC2 instance**
Cloud9 creates an EC2 instance in your account. The configuration of your EC2 instance cannot be changed by Cloud9 after creation.

☐ **Existing compute**
You have an existing instance or server that you'd like to use.

4. Configure the Cloud9 Environment

- **Set up the instance type:**
 - In this step, you'll choose the environment configuration.
 - **Instance type:** Select the default instance type `t2.micro`. This is sufficient for this project and is eligible for AWS **Free Tier**, which means you won't incur charges for using it.
- **Configure other settings:**
 - Leave all the other options as default (for example, VPC, network settings, storage, etc.).

5. Review and Create

- **Review your configuration:**
 - **Review the details of your Cloud9 environment setup on this screen. Double-check that everything is set to your needs.**
- **Create the environment:**
 - **Once you've confirmed everything looks good, click Create environment.**
- **Wait for your environment to be provisioned:**
 - **It may take a few minutes for AWS to create and launch your Cloud9 environment.**

New EC2 instance

Instance type [Info](#)
The memory and CPU of the EC2 instance that will be created for Cloud9 to run on.

☒ **t2.micro** (1 GiB RAM + 1 vCPU)
Free-tier eligible. Ideal for educational users and exploration.

☐ **t3.small** (2 GiB RAM + 2 vCPU)
Recommended for small web projects.

☐ **m5.large** (8 GiB RAM + 2 vCPU)
Recommended for production and most general-purpose development.

☐ **Additional instance types**
Explore additional instances to fit your need.

Platform [Info](#)
This will be installed on your EC2 instance. We recommend Amazon Linux 2023.

Amazon Linux 2023 ▼

Timeout
How long Cloud9 can be inactive (no user input) before auto-hibernating. This helps prevent unnecessary charges.

30 minutes ▼

Network settings [Info](#)

Connection

How your environment is accessed.

☐ AWS Systems Manager (SSM)
Accesses environment via SSM without opening inbound ports (no ingress).

☒ Secure Shell (SSH)
Accesses environment directly via SSH, opens inbound ports.

[▶ VPC settings](#) [Info](#)

[▶ Tags - optional](#) [Info](#)

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

 The following IAM resources will be created in your account

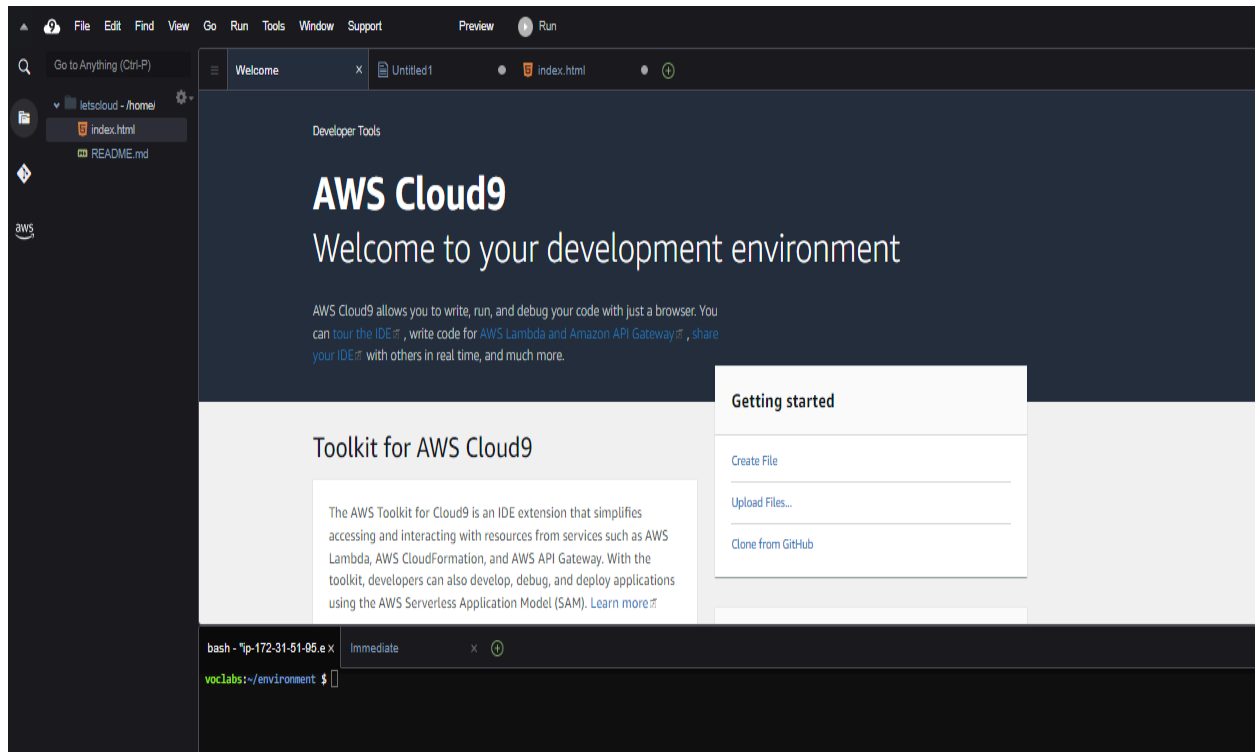
- **AWSServiceRoleForAWSCloud9** - AWS Cloud9 creates a service-linked role for you. This allows AWS Cloud9 to call other AWS services on your behalf. You can delete the role from the AWS IAM console once you no longer have any AWS Cloud9 environments. [Learn more](#) 

Cancel

Create

6. Access Your Cloud9 Environment

- **You'll be redirected to the Cloud9 IDE:**
 - Once the environment is ready, you will automatically be taken to the **Cloud9 IDE** upon click open, which is a web-based development environment.
- **Overview of the Cloud9 IDE:**
 - On the **left**, you'll see a **file explorer** where you can manage and create files.
 - In the **middle**, you have the **code editor** where you will write your HTML code.
 - At the **bottom**, there's a **terminal (command-line)** where you can run commands to manage your environment.



Step 2: Create a Basic HTML Page in Cloud9

1. Create an HTML file

- **Right-click in the file explorer:**
 - In the left panel (the file explorer), right-click on the empty space or on the folder icon and select **New File**.
- **Name the file:**
 - Name your file something like `index.html`. This will be the main page of your website.

Here's a detailed guide to help you complete Task 1: creating a basic HTML page using AWS Cloud9:

Step 1: Set Up AWS Cloud9

1. Log in to your AWS Management Console

- **Visit the AWS Management Console:**
 - Open a web browser and navigate to [AWS Management Console](#).
- **Log in:**
 - If you already have an AWS account, use your credentials to log in.

- If you don't have an AWS account yet, create one by clicking [here](#) and following the steps to set up your account.

Tip: If you're signing up for a new AWS account, you may need to enter your credit card information, but AWS offers a **Free Tier** for many services, including Cloud9.

2. Navigate to AWS Cloud9

- **Find Cloud9 using the search bar:**
 - After logging in, locate the **search bar** at the top of the AWS Management Console.
 - Type **Cloud9** into the search bar, and from the results, click on **Cloud9** under "Developer Tools."
- **Open the Cloud9 service:**
 - This will bring you to the Cloud9 dashboard, where you can manage your Cloud9 environments.

3. Create a New Cloud9 Environment

- **Start by creating an environment:**
 - Click on the **Create environment** button on the top right of the Cloud9 dashboard.
- **Give your environment a name:**
 - In the first step of creating the environment, you will need to **name** it. For example, name it **MyWebsiteEnv** (or any other name you prefer).
 - In the **Description** field, add something like "Environment for hosting my simple HTML page."
- **Proceed to the next step:**
 - Once you've added the name and description, click **Next step**.

4. Configure the Cloud9 Environment

- **Set up the instance type:**
 - In this step, you'll choose the environment configuration.
 - **Instance type:** Select the default instance type **t2.micro**. This is sufficient for this project and is eligible for AWS **Free Tier**, which means you won't incur charges for using it.
- **Configure other settings:**
 - Leave all the other options as default (for example, VPC, network settings, storage, etc.).
 - Click **Next step** to continue.

5. Review and Create

- **Review your configuration:**

- Review the details of your Cloud9 environment setup on this screen. Double-check that everything is set to your needs.
- **Create the environment:**
 - Once you've confirmed everything looks good, click **Create environment**.
- **Wait for your environment to be provisioned:**
 - It may take a few minutes for AWS to create and launch your Cloud9 environment.

6. Access Your Cloud9 Environment

- **You'll be redirected to the Cloud9 IDE:**
 - Once the environment is ready, you will automatically be taken to the **Cloud9 IDE**, which is a web-based development environment.
 - **Overview of the Cloud9 IDE:**
 - On the **left**, you'll see a **file explorer** where you can manage and create files.
 - In the **middle**, you have the **code editor** where you will write your HTML code.
 - At the **bottom**, there's a **terminal (command-line)** where you can run commands to manage your environment.
-

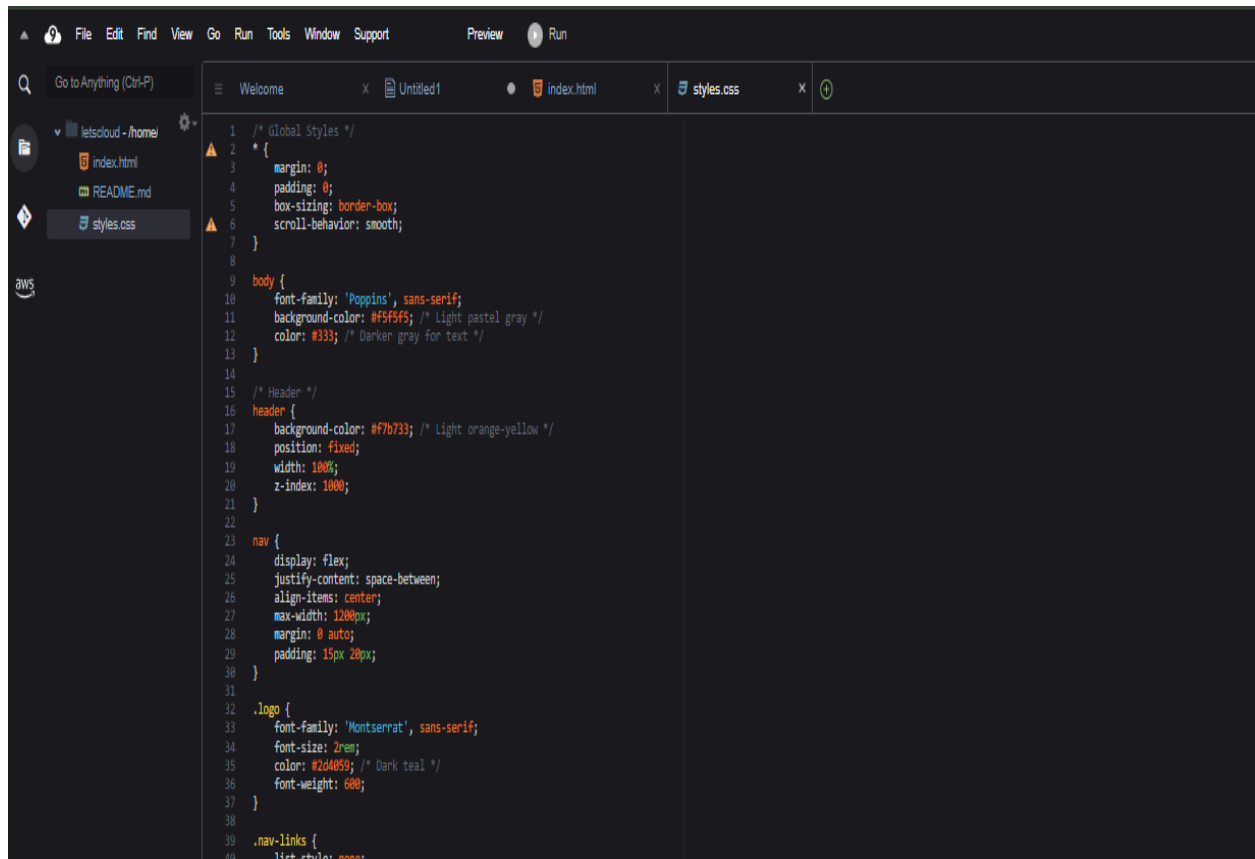
Step 2: Create a Basic HTML Page in Cloud9

1. Create an HTML file

- **Right-click in the file explorer:**
 - In the left panel (the file explorer), right-click on the empty space or on the folder icon and select **New File**.
- **Name the file:**
 - Name your file something like `index.html`. This will be the main page of your website.

2. Write HTML Code

- **Open the file in the editor:**
 - Once the file is created, it will automatically open in the editor. If it doesn't, double-click on the `index.html` file to open it.
- **Write your HTML code:**
 - In the code editor, enter the following basic HTML code to create a simple webpage:



:

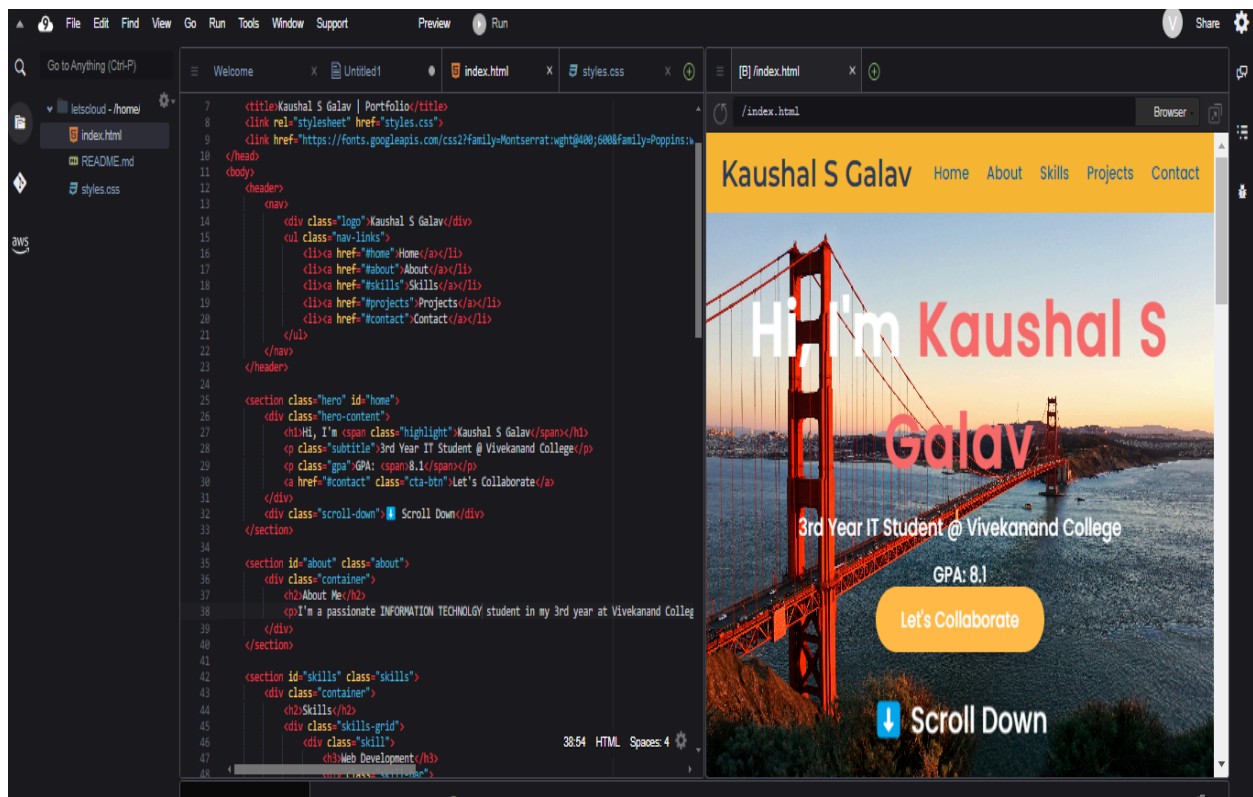
3. Save the file

- **Save your work:**
 - Once you've written the HTML code, make sure to save the file. You can do this by either pressing **Ctrl + S** (Windows) or **Cmd + S** (Mac) or by clicking on the **File** menu and selecting **Save**.

Step 3: Preview Your HTML Page

1. Preview the HTML page in Cloud9

- **Use the Preview feature:**
 - To view the page you've just created, click on the **Preview** menu at the top of the Cloud9 interface and select **Preview Running Application**.
- **See your website:**
 - This will open a new browser tab or a preview pane in the Cloud9 IDE, where you can see your simple HTML page with the heading "Welcome to My First Website!" and the paragraph you added.
- **Alternatively, view in a new tab:**
 - You can also open the preview in a new browser tab by clicking the small arrow next to the preview button and selecting **Preview in New Tab**. This gives you a full view of your webpage.



Step 4: Downloading Your HTML Page from Cloud9

1. Download the `index.html` File:

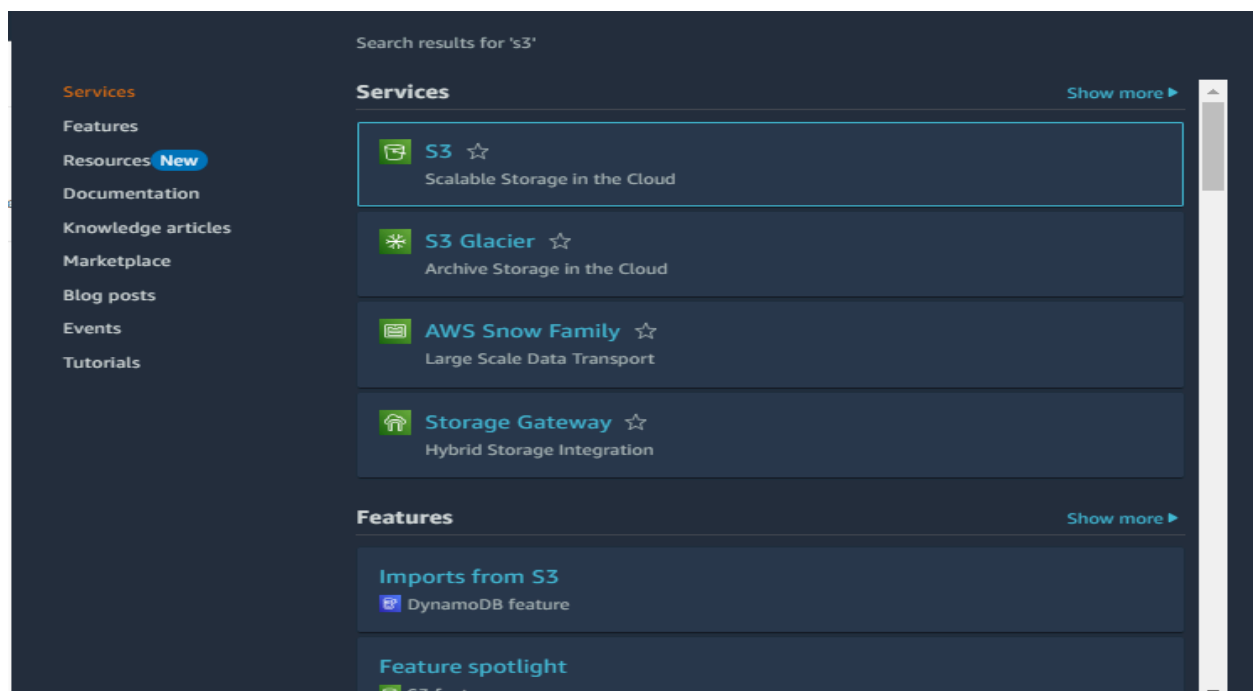
- In the Cloud9 IDE, navigate to the file explorer panel on the left.
- Right-click on the `index.html` file.
- Select Download. This will save the file to your local computer, typically in your default downloads folder.

Task 2: Deploy the HTML Page to an Amazon S3 Bucket

Step 1: Create an S3 Bucket

1. Access the S3 Service:

- Log in to your AWS Management Console.
- In the **top search bar**, type **S3** and press **Enter** or click on it in the dropdown list. This will take you to the Amazon S3 console, where you can manage your S3 buckets.



Create a New S3 Bucket:

- Click on the **Create bucket** button, usually located at the top right corner of the S3 dashboard.
- **Bucket Name:** Enter a unique name for your bucket. For example, you can use `my-simple-website-bucket-unique`. **Note:** Bucket names must be unique across all AWS users, so ensure that your chosen name is not already taken.
- **Region:** Select the AWS region where you want your bucket to reside (e.g., US East (N. Virginia)). Choosing a region closer to your target audience can reduce latency.
- **Object Ownership:** Keep it as **ACLs disabled** (recommended) for simplicity and security.
- **Block Public Access Settings:** **Uncheck** the option that says **Block all public access**. This will allow you to set permissions later to make your website publicly accessible.
- Rest all are **default** settings.

[Amazon S3](#) > [Buckets](#) > Create bucket

Create bucket [Info](#)

Buckets are containers for data stored in S3.

General configuration

AWS Region
US East (N. Virginia) us-east-1

Bucket type [Info](#)

☒ **General purpose**
Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.

☐ **Directory**
Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.

Bucket name [Info](#)

Bucket name must be unique within the global namespace and follow the bucket naming rules. [See rules for bucket naming](#) [?](#)

Copy settings from existing bucket - *optional*
Only the bucket settings in the following configuration are copied.

Format: s3://bucket/prefix

Object Ownership [Info](#)

Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

☒ ACLs disabled (recommended)

All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.

☐ ACLs enabled

Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.

Object Ownership

Bucket owner enforced

Block Public Access settings for this bucket

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

☐ Block *all* public access

Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

☐ Block public access to buckets and objects granted through *new* access control lists (ACLs)

S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.

☐ Block public access to buckets and objects granted through *any* access control lists (ACLs)

S3 will ignore all ACLs that grant public access to buckets and objects.

Locate Your Bucket:

- After clicking **Create bucket**, you will be taken back to the S3 dashboard. Your new bucket will now be listed. Click on its name to enter the bucket's settings page.

General purpose buckets (4) [Info](#) [All AWS Regions](#)

Buckets are containers for data stored in S3.

[Refresh](#) [Copy ARN](#) [Empty](#) [Delete](#) [Create bucket](#)

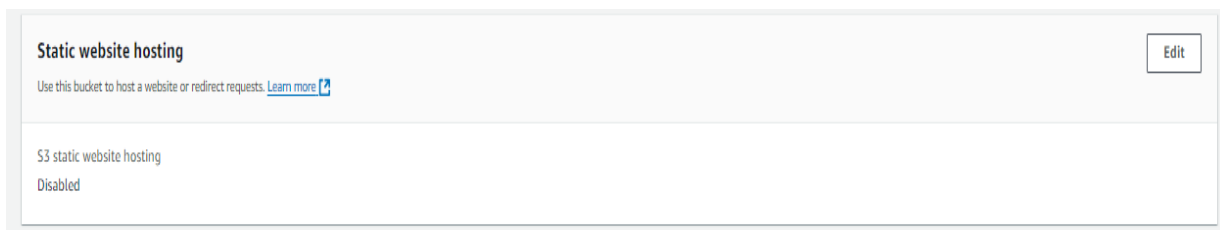
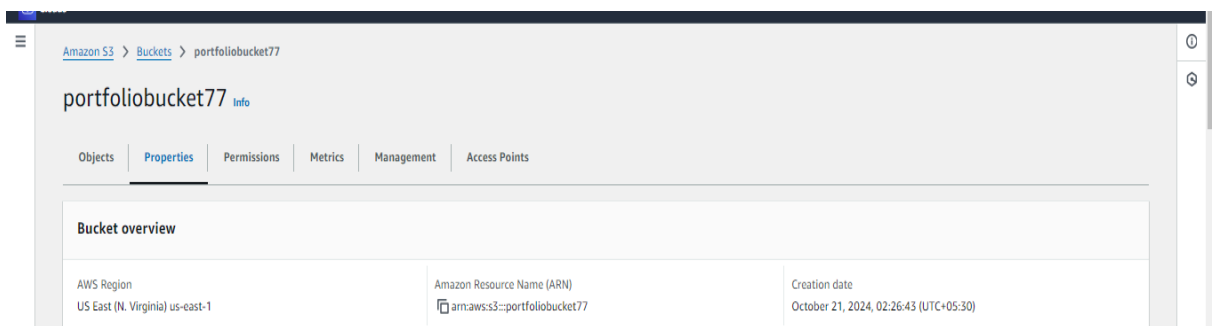
< 1 > [Settings](#)

Name	AWS Region	IAM Access Analyzer	Creation date
☐ aplibucket89	US East (N. Virginia) us-east-1	View analyzer for us-east-1	October 18, 2024, 21:47:42 (UTC+05:30)
☐ elasticbeanstalk-us-east-1-945236649319	US East (N. Virginia) us-east-1	View analyzer for us-east-1	August 7, 2024, 09:47:19 (UTC+05:30)
☐ kaushalexp12bucket	US East (N. Virginia) us-east-1	View analyzer for us-east-1	October 12, 2024, 10:40:04 (UTC+05:30)
☐ portfoliobucket77	US East (N. Virginia) us-east-1	View analyzer for us-east-1	October 21, 2024, 02:26:43 (UTC+05:30)

Step 2: Configure Static Website Hosting

1. Enable Static Website Hosting:

- In your bucket settings, find the **Properties** tab. Click on it to view various configuration options.
- Scroll down until you see the **Static website hosting** section.
- Click on **Edit** to modify the settings.
- Select the **Enable** option for static website hosting.
- **Index Document:** Enter `index.html` in this field. This tells S3 that `index.html` is the default page to display when someone accesses the root of your website.
- **Error Document:** You can enter `index.html` or leave this field blank. If you want, this can serve as a catch-all for errors, redirecting users to the homepage.
- After filling in these details, click **Save changes** at the bottom.



Static website hosting

Use this bucket to host a website or redirect requests. [Learn more](#)

Static website hosting

☐ Disable
 ☒ Enable

Hosting type

☒ Host a static website
 Use the bucket endpoint as the web address. [Learn more](#)

☐ Redirect requests for an object
 Redirect requests to another bucket or domain. [Learn more](#)

For your customers to access content at the website endpoint, you must make all your content publicly readable. To do so, you can edit the **S3 Block Public Access** settings for the bucket. For more information, see [Using Amazon S3 Block Public Access](#)

Index document

Specify the home or default page of the website.

Error document - optional

This is returned when an error occurs.

Redirection rules - optional

Redirection rules, written in JSON, automatically redirect webpage requests for specific content. [Learn more](#)

1	

Cancel
Save changes

Step 3: Set Bucket Policy for Public Access

1. Adjust Permissions:

- Still in the bucket settings, go to the **Permissions** tab.
- Scroll down to find **Bucket policy** and click on **Edit**.
- To allow public access to your HTML page, you will need to add a bucket policy. Copy and paste the following JSON policy into the policy editor, making sure to replace `my-simple-website-bucket-unique` with your actual bucket name:

portfoliobucket77
 Info

Objects
Properties
Permissions
Metrics
Management
Access Points

Permissions overview

Bucket policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::my-simple-website-bucket-unique/*"
    }
  ]
}
```

Edit bucket policy [Info](#)

Bucket policy

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts. [Learn more](#) 

Bucket ARN

 arn:aws:s3:::portfoliobucket77

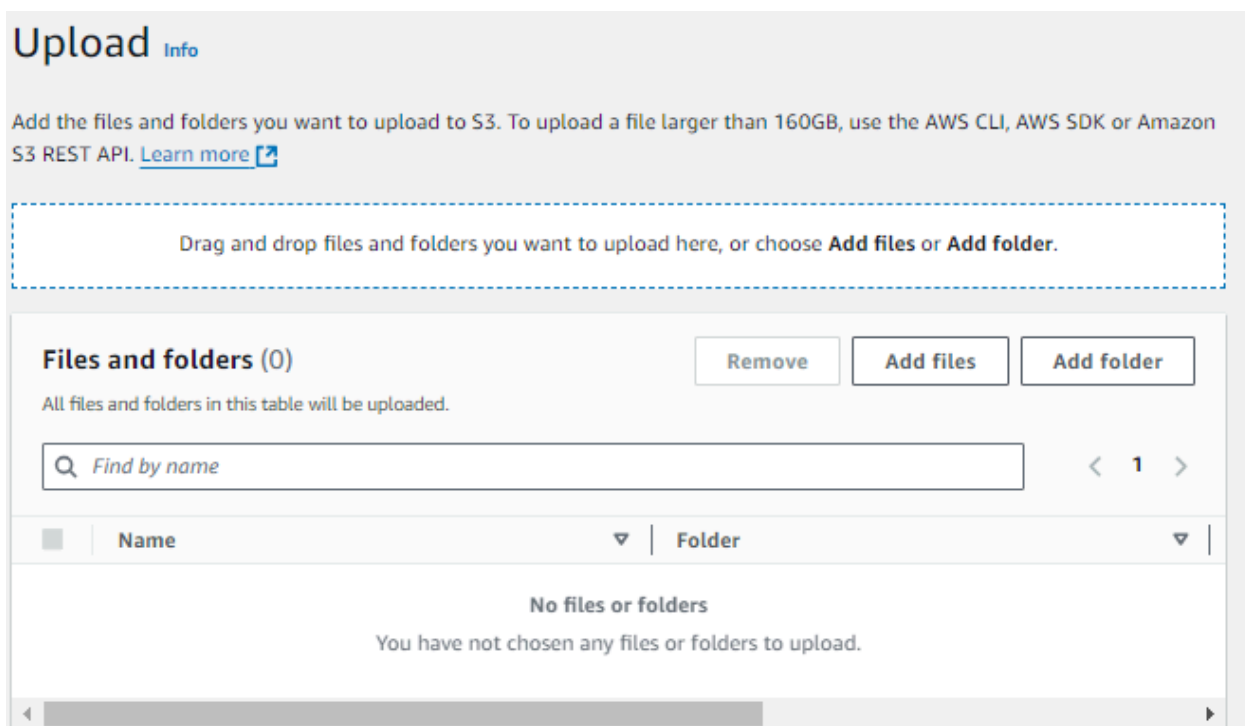
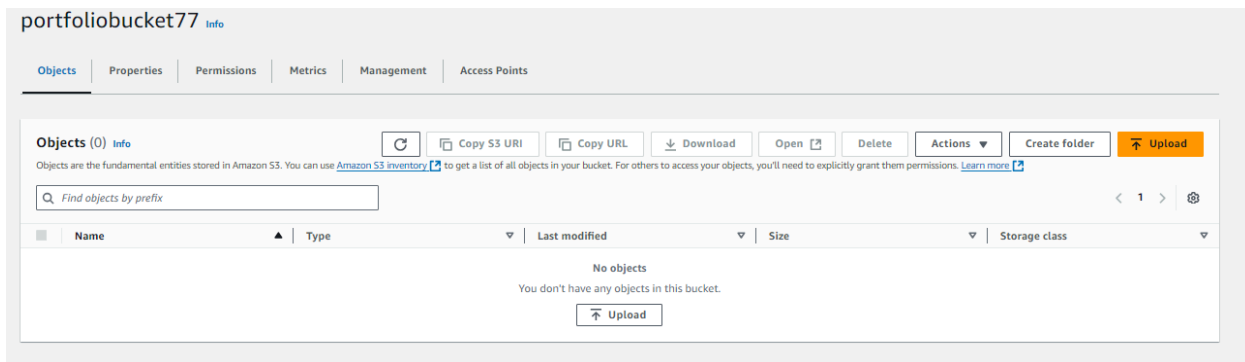
Policy

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "PublicReadGetObject",
6       "Effect": "Allow",
7       "Principal": "*",
8       "Action": "s3:GetObject",
9       "Resource": "arn:aws:s3:::my-simple-website-bucket-unique/*"
10    }
11  ]
12 }
13
```

Step 4: Upload Your HTML File

1. Upload Your `index.html` File:

- With your bucket still open, click on the **Objects** tab to see the current contents of the bucket (which will initially be empty).
- Click on the **Upload** button.
- **Add files:** Click on **Add files** and navigate to where you downloaded your `index.html` file from the Cloud9 environment. Select the file and click **Open**.
- You will see the file listed in the S3 upload interface.



Files and folders (2 Total, 10.1 KB)

Remove

Add files

Add folder

All files and folders in this table will be uploaded.

Q

Find by name

<

1

>

☐	Name	Folder
☐	styles.css	-
☐	index.html	-

Review and Upload:

- Click **Upload** to complete the process. After a few moments, you will see a success message, and your `index.html` file will be listed in your S3 bucket.

Cancel

Upload

Upload succeeded

View details below.

The information below will no longer be available after you navigate away from this page.

Summary

Destination

s3://portfoliobucket77

Succeeded

2 files, 10.1 KB (100.00%)

Failed

0 files, 0 B (0%)

Files and folders

Configuration

Files and folders (2 Total, 10.1 KB)

Find by name

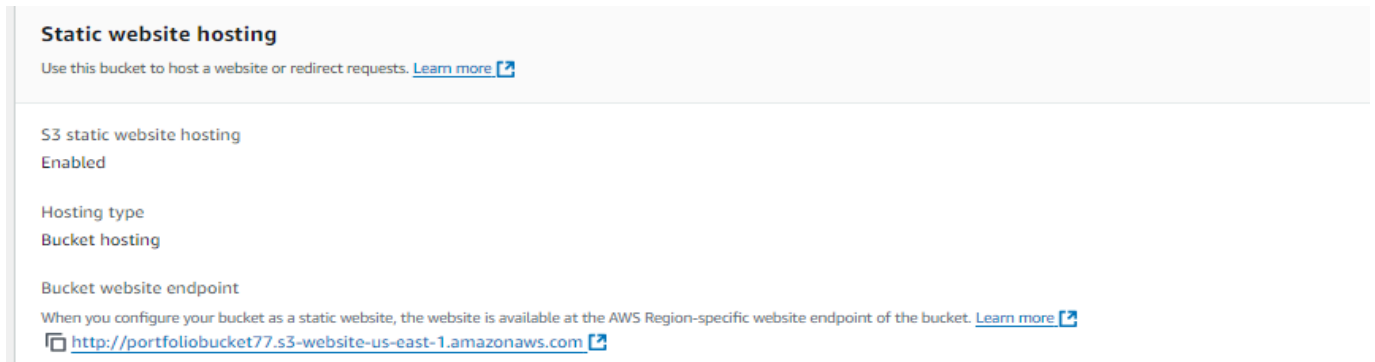
< 1 >

Name	Folder	Type	Size	Status	Error
styles.css	-	text/css	4.9 KB	Succeeded	-
index.html	-	text/html	5.1 KB	Succeeded	-

Step 5: Access Your Static Website

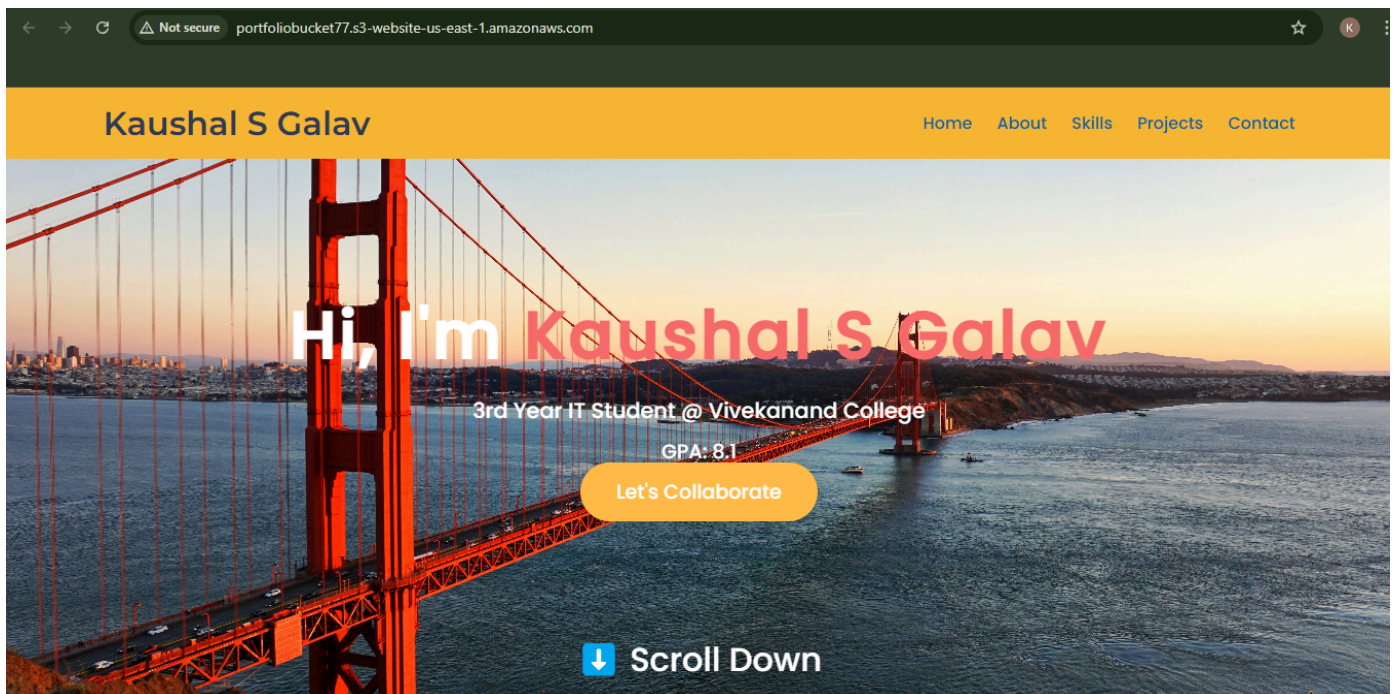
1. Get the Website URL:

- Return to the **Properties** tab of your S3 bucket.
- Scroll down to the **Static website hosting** section. Here, you will find the **Endpoint** URL. This URL is your website's address.



2 Open Your Website:

- Copy the endpoint URL and paste it into your web browser's address bar.
- Press **Enter** to navigate to your static website. You should see the HTML page you created displaying your website which is being hosted on AWS.



Summary

- You have successfully created an S3 bucket, configured it for static website hosting, set the appropriate permissions, uploaded your HTML file, and accessed your website.

Task 3 :Launch an EC2 instance and configure it to serve the same HTML page as a backup.

Step 1: Launch an EC2 Instance

1. **Access EC2:**
 - Log in to your **AWS Management Console**.
 - Search for **EC2** and select it from the dropdown list.
2. **Launch Instance:**
 - Click on the **Launch Instance** button.
3. **Choose an Amazon Machine Image (AMI):**
 - Select **Amazon Linux 2 AMI** or **Ubuntu Server**.
4. **Choose an Instance Type:**
 - Select **t2.micro**.
5. **Configure Security Group:**
 - Click **Add Rule** and configure:
 - **Type:** HTTP
 - **Source:** Anywhere (0.0.0.0/0)
6. **Review and Launch:**
 - Click **Launch**.
 - Select or create a **Key Pair**, download it, and click **Launch Instances**.
7. **View Instances:**
 - Click **View Instances** to see your instance status. Wait for it to show as **running**.

Launch an instance [Info](#)

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags [Info](#)

Name

portfoliobackup

[Add additional tags](#)

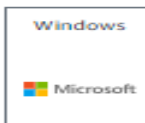
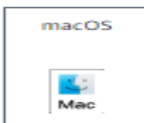
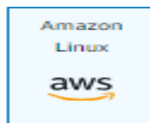
▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below

Search our full catalog including 1000s of application and OS images

Recents

Quick Start



Browse more AMIs
Including AMIs from
AWS, Marketplace and
the Community

▼ Instance type [Info](#) | [Get advice](#)

Instance type

t2.micro

Free tier eligible

Family: t2 1 vCPU 1 GiB Memory Current generation: true
On-Demand Windows base pricing: 0.0162 USD per Hour
On-Demand SUSE base pricing: 0.0116 USD per Hour
On-Demand RHEL base pricing: 0.026 USD per Hour
On-Demand Linux base pricing: 0.0116 USD per Hour

☒ All generations

[Compare instance types](#)

[Additional costs apply for AMIs with pre-installed software](#)

▼ Key pair (login) [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - *required*

key

[Create new key pair](#)

ami
Vir
t2
Fin
Ne
Stc
1 v

▼

Network settings

Info

Edit

Network

Info

vpc-0c9be5ca9875af92d

Subnet

Info

No preference (Default subnet in any availability zone)

Auto-assign public IP

Info

Enable

Additional charges apply when outside of free tier allowance

Firewall (security groups)

Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group

☐ Select existing security group

We'll create a new security group called 'launch-wizard-19' with the following rules:

☒ Allow SSH traffic from

Helps you connect to your instance

Anywhere

0.0.0.0/0

☒ Allow HTTPS traffic from the internet

To set up an endpoint, for example when creating a web server

☒ Allow HTTP traffic from the internet

To set up an endpoint, for example when creating a web server

⚠

Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

×

Cancel

Launch instance

📄

Preview code

☐	aws-cloud9-le...	i-024245d4ce8e0aa55	✔ Running	🔍 🔍	t2.micro	✔ 2/2 checks passed	View alarms +
☒	portfoliobackup	i-0eaa1a039ad44dff8	✔ Running	🔍 🔍	t2.micro	⌚ Initializing	View alarms +

Step 2: Connect to Your EC2 Instance

- Get the Public IP:
 - Select your instance and note the **Public IPv4 address**.
- Connect via SSH:
 - Open a terminal (Linux/Mac) or Command Prompt/PowerShell (Windows).
 - Navigate to the directory where your key pair `.pem` file is saved.

On terminal:

- Change permissions of the `.pem` file (if using Unix/Linux/Mac):

```
chmod 400 MyKeyPair.pem
```

- Connect to your instance using SSH (for Amazon Linux):

```
ssh -i "MyKeyPair.pem" ec2-user@your-instance-public-ip
```

Step 3: Install Apache on Your EC2 Instance

1. Update Package Repository:

- For **Amazon Linux**:

```
sudo yum update -y
```

2. Install Apache:

- For **Amazon Linux**:

```
sudo yum install httpd -y
```



```
PS C:\Users\Admin> cd downloads
PS C:\Users\Admin\downloads> ssh -i "key.pem" ec2-user@ec2-54-209-217-99.compute-1.amazonaws.com
The authenticity of host 'ec2-54-209-217-99.compute-1.amazonaws.com (54.209.217.99)' can't be established.
ED25519 key fingerprint is SHA256:l3fIN4WbNqbcWFA8xJRlD9G+dVGt60dDUDfCFHSrXm2o.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-54-209-217-99.compute-1.amazonaws.com' (ED25519) to the list of known hosts.
```

```

#_
~\_ ##### Amazon Linux 2023
NN \#####\
NN \###|
NN \#/ --- https://aws.amazon.com/linux/amazon-linux-2023
NN V~' '->
NNN
NN .-./
-./
-/m/'

```

```
[ec2-user@ip-172-31-46-171 ~]$ sudo yum update -y
Last metadata expiration check: 0:15:20 ago on Sun Oct 20 21:49:23 2024.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-46-171 ~]$ sudo yum install httpd -y
Last metadata expiration check: 0:15:43 ago on Sun Oct 20 21:49:23 2024.
Dependencies resolved.
```

Package	Architecture	Version	Repository	Size
Installing:				
httpd	x86_64	2.4.62-1.amzn2023	amazonlinux	48 k

3.Start Apache Service:

- For Amazon Linux:
`sudo systemctl start httpd`

4.Enable Apache on Boot:

- For Amazon Linux:
`sudo systemctl enable httpd`

```
Complete!
[ec2-user@ip-172-31-46-171 ~]$ sudo systemctl start httpd
[ec2-user@ip-172-31-46-171 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[ec2-user@ip-172-31-46-171 ~]$ |
```

Step 4: Upload Your HTML File to the EC2 Instance

Use the `scp` command to upload your `index.html` file. Adjust the paths and instance details accordingly.

```
scp -i "C:\path\to\your\key.pem" "C:\Users\Admin\Downloads\index.html"
ec2-user@ec2-18-232-166-146.compute-1.amazonaws.com:/home/ec2-user/
```

```
PS C:\Users\Admin\downloads> scp -i key.pem "C:\Users\Admin\Downloads\index.html" ec2-user@54.209.217.99:/home/ec2-user/
index.html                                     100% 5236   14.1KB/s   00:00
PS C:\Users\Admin\downloads> scp -i key.pem "C:\Users\Admin\Downloads\styles.css" ec2-user@54.209.217.99:/home/ec2-user/
styles.css                                    100% 5061   16.6KB/s   00:00
PS C:\Users\Admin\downloads> |
```

3. SSH into Your EC2 Instance

```
ssh -i "C:\path\to\your\key.pem"
ec2-user@ec2-18-232-166-146.compute-1.amazonaws.com
```

4. Move Files to the Apache Directory

```
sudo mv /home/ec2-user/index.html /var/www/html/
```

5. Set Permissions (if necessary)

```
sudo chmod 644 /var/www/html/index.html
```

```
PS C:\Users\Admin\downloads> ssh -i "key.pem" ec2-user@ec2-54-209-217-99.compute-1.amazonaws.com

#_
#####_
nn\#####\
nn\###|
nn\#/
nnV~' -> https://aws.amazon.com/linux/amazon-linux-2023

Last login: Sun Oct 20 22:24:22 2024 from 42.107.82.25
[ec2-user@ip-172-31-46-171 ~]$ sudo mv /home/ec2-user/index.html /var/www/html/index.html
sudo mv /home/ec2-user/styles.css /var/www/html/styles.css
[ec2-user@ip-172-31-46-171 ~]$ sudo chown apache:apache /var/www/html/index.html
sudo chown apache:apache /var/www/html/styles.css
[ec2-user@ip-172-31-46-171 ~]$ sudo chmod 644 /var/www/html/index.html
sudo chmod 644 /var/www/html/styles.css
```

6. Verify Apache is Running

Check Apache status:

```
sudo systemctl status httpd
```

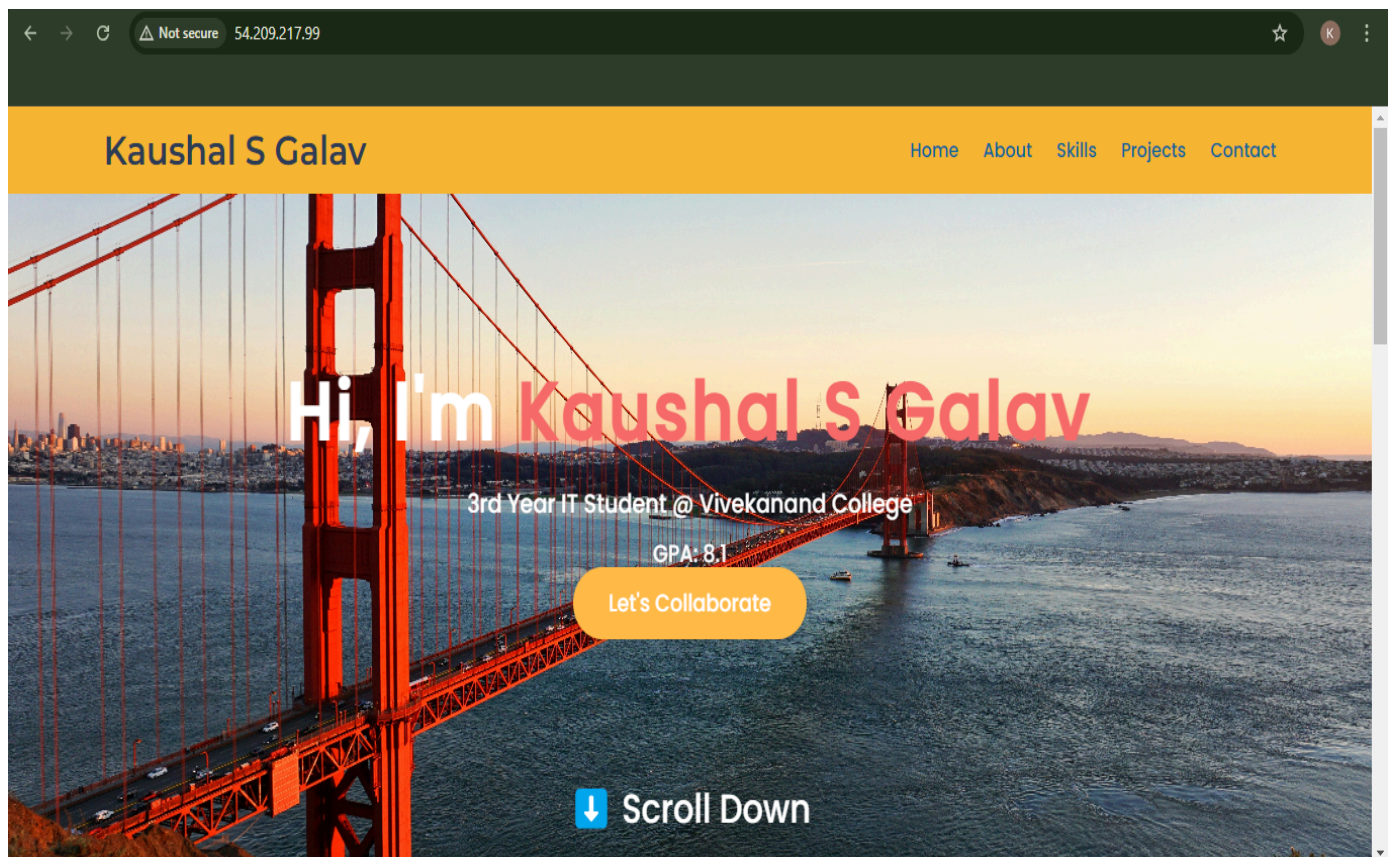
If not running, start it:

```
sudo systemctl start httpd
```

6. Access Your Website

Open your browser and go to:

<http://your-public-ip>



Summary

You have successfully launched an EC2 instance, connected via SSH, installed Apache, uploaded your HTML file, and tested your web server.

Conclusion

In this project, we created a basic HTML page and deployed it using AWS services. We started by designing the webpage in AWS Cloud9. Then, we uploaded it to an S3 bucket and enabled static website hosting, allowing users to access the page directly via a public URL. Finally, we launched an EC2 instance to serve as a backup for the HTML page, ensuring that the content remains available even if the primary hosting method encounters issues. This setup highlights effective use of AWS for hosting web content reliably and efficiently.

